Assignment 1 Writeup——Hao Wang

1. Motivation

This project asks a simple question: can structured information extracted from earnings call transcripts help predict short-horizon stock performance after the call?

My approach was to build a full pipeline from raw transcript text to model predictions and backtests. The goal was to test whether LLM-based transcript features could be made structured, interpretable, and useful enough to show signal in a small academic setting.

The final workflow parses transcripts, extracts structured information with an LLM, converts the extraction into numeric features, aligns those features with future returns, trains prediction models, and evaluates the results with a daily event-driven backtest.

For presentation and inspection, I also keep a notebook called `project walkthrough.ipynb`, which reads the saved artifacts and gives a quick walkthrough of features, predictions, and final report outputs.

2. Data and Setup

The dataset used in the current run contains 131 earnings calls from 14 tickers, covering January 2024 through April 2026. I used a per-ticker chronological split, taking the first 6 calls of each ticker as training data and the remaining calls as test data. That leaves a fairly small supervised learning problem, which shapes nearly every design choice in the project.

The prediction target in the main regression setup is `y_excess`, defined as the stock's forward return minus the benchmark return over a short holding window. The benchmark is `SPY`. For classification models, I discretized the same return measure into three classes: up, flat, and down.

I did not standardize features. Most of the final features are already bounded or fairly close in scale, and I preferred to keep them interpretable in their native units. Given the sample size, I thought simplicity was more important than adding another preprocessing layer.

3. Extraction Design

The LLM extraction prompt produces one unified JSON object with three sections:

- `call_level`
- `speaker_level`
- `reactive_level`

This split was intentional. I wanted the extraction to reflect how earnings calls actually work. The call-level block captures the overall tone, wins, risks, guidance changes, and themes. The speaker-level block separates CEO, CFO, analyst, prepared-remarks, and Q&A sentiment. The reactive-level block distinguishes what management chose to talk about proactively from what only appeared when analysts pushed on it.

I also used a controlled topic vocabulary. That makes the outputs much more stable across quarters and helps downstream feature construction. Without a controlled set of themes, the same idea can easily be labeled in slightly different ways from one transcript to the next.

The prompt is strict in structure and asks for JSON only. The purpose was not to make the extraction perfect. The purpose was to make it consistent enough for repeated experiments.

4. Features

The final NLP feature set includes:

- overall sentiment
- CEO/CFO consistency
- analyst sentiment
- reactive sentiment
- net guidance score
- reactive topic ratio
- average risk sentiment
- risk persistence ratio
- new risk flag
- quarter-over-quarter sentiment delta
- quarter-over-quarter guidance delta
- new theme ratio

I also added three external price-volume features:

- 5-day average return before the call
- MACD-style momentum
- volume divided by ADV20

These external features were added so I could compare transcript-only models with transcript-plus-market-context models.

One thing that did not work well was over-splitting sentiment. More granular features like `qa_minus_prepared_sentiment` and `avg_win_sentiment` were too correlated with overall sentiment, so they added complexity faster than they added useful information.

5. Backtest Design

The backtest went through an important revision.

My first attempt was a very simple event-level backtest. After each earnings call, I computed the return from the entry date to the event-specific exit date, multiplied that by the model signal, and then built an equity curve by chaining those event returns with `cumprod`. That is easy to code, but it is not a realistic portfolio simulation.

The problem is that this approach implicitly treats trades as if they happen one after another. In the real strategy, holdings overlap. If one earnings-call trade enters on Monday and another enters on Wednesday, both positions should be alive at the same time until their own exit dates. A pure event-level `cumprod` ignores that overlap, hides the true capital usage, and can make the path of returns look cleaner than a tradable strategy would actually be.

So I changed the backtest to a daily event-driven portfolio construction that updates holdings through time. The logic is:

- each qualifying event opens a fixed-size long or short position at `entry_date`
- position size is explicit: each active trade contributes one unit of notional capital, so `position = +1` for a long and `position = -1` for a short
- the position stays open until its own `exit_date`, even if newer signals arrive in the meantime
- on each day, portfolio return is computed using all positions that are active on that date

The daily strategy return is:

`sum(position * stock_daily_return) / sum(abs(position))`

The denominator makes the exposure normalization explicit: if there are three active trades, the strategy is averaging across three units of gross exposure; if there is only one active trade, the return comes from that single unit. If there are no active positions on a day, the strategy return is zero. The benchmark is tracked separately using its own daily return series.

This construction is much closer to what the strategy would look like in practice. It gives the equity curve a real calendar-time axis, handles overlapping holdings correctly, and makes the portfolio interpretation much clearer than the original event-level `cumprod` shortcut.

6. Models

I focused the final comparison on four models:

- `ridge_reg`
- `xgb_reg`
- `logistic_clf`
- `xgb_clf`

For each model, I mainly compare:

- `nlp_only`
- `nlp_plus_external`

I did train `external_only` versions during development, but I treat them as diagnostic baselines rather than main results.

I did not run a real hyperparameter search. With this sample size, a large parameter grid would almost certainly overfit. Instead, I manually chose relatively small and regularized models. This is not ideal, but I think it is more honest than pretending a highly tuned model would be reliable in a dataset this small.

7. Main Results

Before looking at the model comparison tables, it is useful to state the target definition and backtest settings used for the reported results. The target window in `default.yaml` is `window_size = 5`, so returns are measured from the next trading day after the call to the close 5 trading days later. Because `use_excess = true`, the regression target is `y_excess`, meaning 5-day stock return minus 5-day SPY return. Because `use_classification = true`, the classification target is built from that same excess-return measure with `classification_up_threshold = 0.01` and `classification_down_threshold = -0.01`, so labels are `up`,

`flat`, or `down` depending on whether 5-day excess return is above 1%, between -1% and 1%, or below -1%. For the trading results, I used the default backtest parameters: `signal_col = y_pred`, `greater_is_better = true`, `long_threshold = 0.0`, `short_threshold = -0.0`, and `annualization_days = 252`. In practical terms, that means predictions above zero are treated as long signals, predictions below zero are treated as short signals, and there is effectively no neutral buffer zone around zero. Reported mean return and Sharpe are annualized using 252 trading days.

7.1 Prediction Metrics

The table below summarizes the main prediction-level results for the four highlighted models.

Model	Feature Set	Main Metric	Result
`ridge_reg`	`nlp_only`	Spearman IC	`0.168`
`ridge_reg`	`nlp_plus_external`	Spearman IC	`0.190`
`xgb_reg`	`nlp_only`	Spearman IC	`0.128`
`xgb_reg`	`nlp_plus_external`	Spearman IC	`0.057`
`logistic_clf`	`nlp_only`	Accuracy / IC	`0.432 / 0.198`
`logistic_clf`	`nlp_plus_external`	Accuracy / IC	`0.386 / -0.070`
`xgb_clf`	`nlp_only`	Accuracy / IC	`0.432 / 0.054`
`xgb_clf`	`nlp_plus_external`	Accuracy / IC	`0.409 / 0.000`

At the prediction level, `ridge_reg_nlp_plus_external` has the strongest regression IC, while `logistic_clf_nlp_only` has the strongest classification IC. That already suggests the data contains some weak ranking signal, but not enough to make all model classes equally useful.

7.2 Backtest Metrics

The table below summarizes the daily event-backtest results for the same eight model variants.

Model	Feature Set	Mean Return	Sharpe	Directional Accuracy
`ridge_reg`	`nlp_only`	`0.125`	`0.270`	`0.523`
`ridge_reg`	`nlp_plus_external`	`0.008`	`0.018`	`0.500`
`xgb_reg`	`nlp_only`	`0.309`	`0.714`	`0.568`
`xgb_reg`	`nlp_plus_external`	`0.595`	`1.305`	`0.477`
`logistic_clf`	`nlp_only`	`1.026`	`2.841`	`0.516`
`logistic_clf`	`nlp_plus_external`	`0.302`	`0.898`	`0.423`
`xgb_clf`	`nlp_only`	`0.208`	`0.458`	`0.432`

Model	Feature Set	Mean Return	Sharpe	Directional Accuracy
`xgb_clf`	`nlp_plus_external`	`0.614`	`1.363`	`0.409`

These numbers are more interesting than the raw prediction metrics. The strongest result in the current run is `logistic_clf_nlp_only`, which has the highest annualized mean return and the highest Sharpe. That is a genuinely good result relative to the rest of the model set.

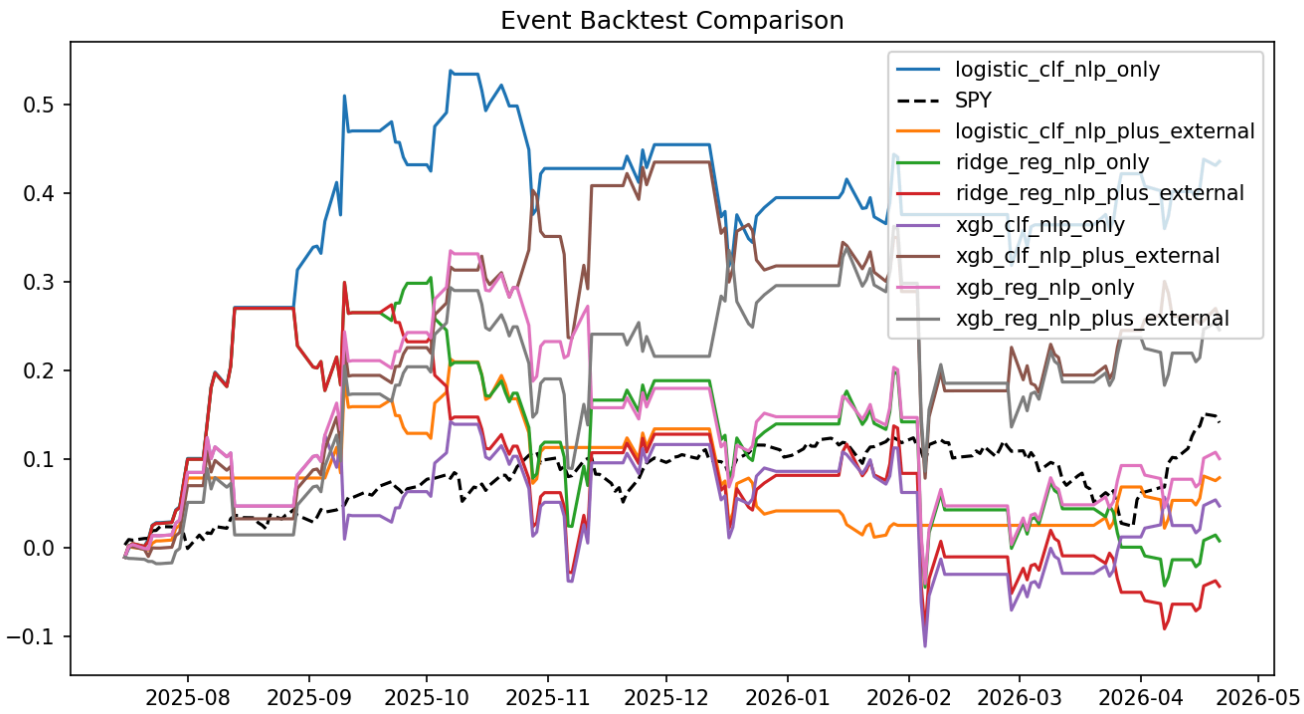
The best tree-based regression backtest is `xgb_reg_nlp_plus_external`. It is noticeably stronger than `xgb_reg_nlp_only` in backtest terms, even though its information coefficient is weaker. That suggests the external features may be helping on a subset of actual trading decisions rather than improving the full ranking problem.

Ridge looks more stable at the prediction level than in backtest. `ridge_reg_nlp_plus_external` has the best regression IC overall, but its backtest is basically flat. `ridge_reg_nlp_only` is weaker in prediction ranking but better in realized strategy performance. That mismatch is a useful reminder that statistical ranking quality and backtest quality are not identical.

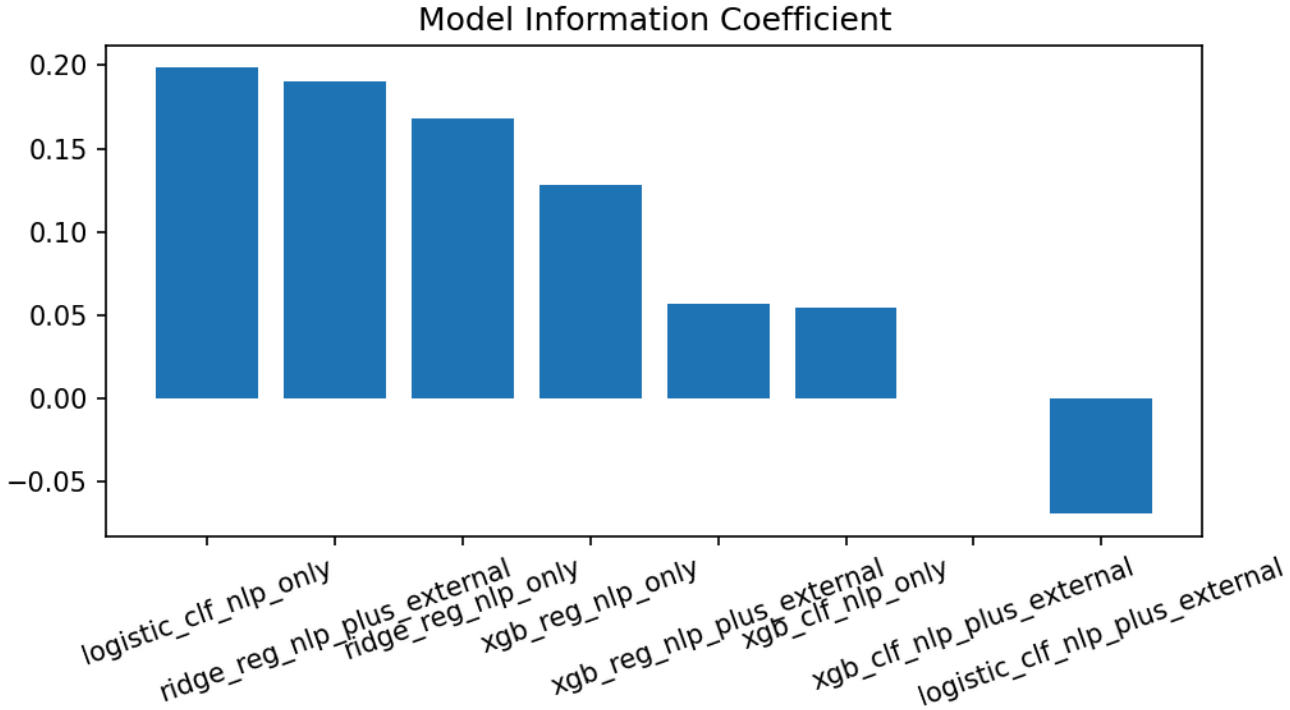
For the classifiers, `logistic_clf_nlp_only` is clearly stronger than `logistic_clf_nlp_plus_external`. In contrast, `xgb_clf_nlp_plus_external` looks better than `xgb_clf_nlp_only` in backtest terms, but the signal still does not look fully convincing because its IC is essentially zero. I would treat that result cautiously.

7.3 Figures

The main model-comparison equity curve is shown below.



The IC bar chart is also useful because it makes the ranking-quality differences easier to see than the equity curves alone.



The equity curve comparison tells a fairly balanced story. Some variants do beat the benchmark over this sample, and some do so by a meaningful amount. At the same time, the gap between models is large enough that I do not think it is safe to say the pipeline has found a robust and model-independent signal. The results are promising in places, but not stable enough to be called strong evidence.

8. Comparing Feature Sets

The feature comparison does not support one simple conclusion.

For `ridge_reg`, adding the external features does not help. The combined model slightly improves IC, but the backtest gets worse. That makes me think the external block does not add much useful linear information on top of the NLP features here.

For `xgb_reg`, the combined version is better in backtest terms even though its IC is lower. My interpretation is that the small amount of market context may be helping the model choose a few more useful trades, even if the overall ranking of all names is not better.

For `logistic_clf`, the pure NLP version is clearly better. This is probably the cleanest result in the whole project: transcript features alone work better than transcript-plus-external features for this particular linear classifier.

For `xgb_clf`, the combined version has the better backtest, but because its IC is zero I would not overstate that result. It may be capturing threshold effects rather than a meaningful ranking signal.

So the answer is not "external signals always help" and not "NLP alone is always enough." The effect depends on the model, and with this sample size those conclusions should be treated as tentative.

9. What the Speaker and Reactive Features Added

Even where the final backtest is weak, I still think the speaker-level and proactive-versus-reactive features were worthwhile.

The speaker-level information helps separate different voices inside the same call. Prepared remarks often sound positive almost by construction, while analyst questions and reactive answers can be more revealing. That difference is economically meaningful, even if it does not always show up as alpha in a small sample.

The proactive/reactive distinction adds another useful layer. If management highlights a topic in prepared remarks, that is different from a topic that only appears after analysts push on it. The latter may reflect defensiveness, lower emphasis, or information management. Features like `reactive_topic_ratio` were designed to capture exactly that.

My view is that these features improve the representation of the transcript, even when they do not always improve the final trading result.

10. Per-Ticker Qualitative Read

I manually reviewed several transcripts and the corresponding extraction outputs. AMD, JPM, and PLTR were especially useful examples.

AMD

AMD is a good example of why a strong call does not automatically imply positive excess return. The 2025 calls are very upbeat. The extraction highlights record revenue, strong AI demand, and raised guidance. At the same time, it also captures real concerns, including China-related shipment exclusions, higher operating expenses, and weaker gaming trends.

That feels right when reading the call itself. Management sounds confident, but analysts push on real risks. The features capture that tension well. The problem is that the stock can still underperform if expectations were already extremely high going into the call.

JPM

JPM looks more balanced. The extraction is positive, but not euphoric. It picks up strong earnings and better NII guidance, but also deposit margin compression, higher credit costs, and regulatory pressure. The reactive topics in Q&A also look realistic for a large bank.

This is exactly the kind of transcript where speaker-level and reactive features seem useful. The call is not simply “good” or “bad.” It is more like “solid quarter, cautious undertone.” That nuance is valuable even if the stock reaction stays muted.

PLTR

PLTR is probably the clearest case where the extraction looks strong but the investment conclusion is less obvious. The call language is extremely positive, with strong growth, raised guidance, and high margins. CEO and CFO sentiment are both very strong. At the same time, the extraction still picks up reactive issues like supply constraints and uneven geography.

This is a good cautionary example because the model can easily be impressed by the transcript, but the stock may already reflect a lot of that optimism before the call even starts.

11. Limitations

The first limitation is sample size. Once I split by ticker and time, the number of effective observations becomes small very quickly. That alone makes the results fragile.

The second limitation is the ratio of observations to features. Even after simplifying the feature set, there are still enough features to create overfitting risk. That is especially true for flexible tree-based models.

The third limitation is that I did not run a real hyperparameter search. I chose relatively small models by judgment. That was a deliberate decision, but it means the final results can still depend a lot on manual settings.

The fourth limitation is that the external-signal block is still very small. With only a few market variables, it is hard to say much about how useful non-text context really could be. This is one reason I do not think the tree-model results should be over-interpreted.

The fifth limitation is the backtest design choice. The current daily event-driven backtest is a big improvement over a jump-based event return approach, but I still did not tune long/short thresholds or other trading parameters using a proper validation split.

12. What Did Not Work

One thing that did not work was very fine-grained sentiment decomposition. Features like `qa_minus_prepared_sentiment` and `avg_win_sentiment` were too correlated with broader sentiment measures to justify their extra complexity.

Another thing that did not work is robustness under parameter changes. Because the sample is small, the financial target is noisy, and the features are not extremely strong, changing model parameters can lead to noticeably different outcomes. That means the results are not as stable as I would want in a stronger empirical setting.

13. What I Would Do With More Time or More Data

There are three obvious next steps.

First, I would run a proper hyperparameter search, but only after setting aside a real validation split. Right now that would probably overfit, but with more data it would become much more defensible.

Second, I would add more external features. The current market-context block is too small. I would like to include volatility surprise, analyst revision proxies, options information, and simple fundamental trend variables.

Third, I would tune the backtest rules on validation data rather than fixing one set of thresholds. That includes trade thresholds, confidence rules for classifiers, and possibly minimum trade count filters.

I would also be interested in using audio rather than transcript text alone. Vocal hesitation, interruptions, and tone shifts may contain information that the transcript does not preserve.

14. Conclusion

The pipeline does a good job turning raw earnings calls into structured JSON, interpretable features, model outputs, and a backtest that at least has realistic portfolio logic.

Some of the model results are genuinely encouraging, especially `logistic_clf_nlp_only` and `xgb_reg_nlp_plus_external`. At the same time, the whole exercise is constrained by a small sample, noisy targets, weak robustness, and limited external features.

So I would describe the project as a successful first pass. It shows that transcript features can be made systematic and sometimes useful. It does not yet show that they form a stable short-horizon trading signal on their own.